Evaluating Fast Matrix Multiplication Algorithms In Rust

Computational Laboratory Project Report

Alberto Defendi
Project supervisor: Leonardo Robol

July 18, 2026

Abstract

We develop a Rust library to compute the product of two matrices using fast matrix multiplication algorithms for arbitrary CP tensor decompositions. To support general matrix dimensions, dynamic peeling is implemented as a memory-efficient alternative to zero-padding. Performance is evaluated using both Intel MKL `dgemm` (via FFI) and the native Rust library `faer` as the base case. The implementation supports both sequential and parallel execution with multiple scheduling strategies. Finally, benchmarks on an AMD EPYC server demonstrate that fast algorithms outperform standard GEMM base cases for larger matrix sizes.

Contents

1 Introduction

Matrix multiplication is one of the most fundamental computations in numerical linear algebra and scientific computing. We define *fast multiplication algorithms* as those that perform asymptotically fewer floating-point operations, and require asymptotically less data communication than the classical algorithm. In this work, we demonstrate that fast matrix multiplication algorithms are sensitive to the choice of base case, and perform better than their base case for larger sizes. We provide a library function written in Rust to perform matrix multiplication with these algorithms, supporting parallel execution and different general matrix-matrix multiplication (GEMM) libraries. We perform benchmarks using the Intel Math Kernel Library (MKL) `dgemm` (double-precision general matrix-matrix multiplication) and `faer` (a modern, high-performance linear algebra library in Rust). This allows us to evaluate the performance of native Rust kernels integrated into scientific computing stacks, which are frequently chosen for their out-of-the-box ease of use compared to optimized vendor-specific GEMM libraries that require linking configurations. According to Benson and Ballard [1], fast matrix multiplication algorithms have largely been ignored by numerical libraries due to unfounded reasons. As they state, Intel's MKL, AMD's Core Math Library (ACML) do not provide implementations of fast algorithms. In reality, fast matrix multiplication algorithms perform fewer recursive matrix multiplications at the expense of more matrix additions, and are faster than classical ones both in theory and in practice.

Strassen's algorithm is the best-known fast algorithm, and will be at the core of our experiments, but we also support other algorithms among those listed in Table 2. We review those algorithms and methods to construct them in Section 2. In Section 3 we explain the technical details of our implementation, and in Section 4 we explain parallel and algorithmic optimizations. Finally, we summarize experimental results in Section 5. We summarize our findings as follows:

- Performance of the base GEMM library directly impacts the practical performance and scaling crossover point of fast matrix multiplication algorithms.
- We evaluate Rust as a language for high-performance scientific computing, which is currently in the early stages of adoption for sparse and dense matrix operations [2].

1.1 Notation and tensor preliminaries

The relevant notation for our work is in Table 1. We briefly review basic tensor preliminaries, following the notation in [3]. A *tensor* is a multidimensional array, and in this work we deal only with 3-dimensional (or 3-way) tensors $\mathcal{T} \in \mathbb{R}^{M \times N \times P}$. The *k-th frontal slice* of $\mathcal{T}$ is the matrix $\mathbf{X}_k$. For $u \in \mathbb{R}^M, v \in \mathbb{R}^N, w \in \mathbb{R}^P$, we define the rank-1 *outer product* tensor $\mathcal{T} = u \bigcirc v \bigcirc w \in \mathbb{R}^{M \times N \times P}$ with entries $x_{ijk} = u_i v_j w_k$; such tensor has *rank* one. Tensor addition is defined entry-wise. The rank of a tensor

Table 1: Summary of notation.

$\langle m, n, p \rangle$	Bilinear form representing base case matrix multiplication of an $m \times n$ matrix by a $n \times p$ matrix.
$M \times N \times P$	Dimensions of actual matrices that are multiplied ($M \times N$ matrix multiplied by $N \times P$ matrix).
$[\![\mathbf{U}, \mathbf{V}, \mathbf{W}]\!]$	Factor matrices corresponding to a tensor CP decomposition that provides a fast algorithm.
$\mathcal{T}$	Order-3 tensor.
$\mathcal{T} = u \circ v \circ w$	Rank-1 tensor with entries $X_{ijk} = u_i v_j w_k$.
$\mathbf{T}_{(i)}$	ith frontal slice of $\mathcal{T}$, the matrix of entries $X_{:,:,i}$.
$A_{:k}$, $A_{k:}$, A_{ij}	kth column, kth row and i, j entry of matrix $\mathbf{A}$.
$\mathrm{vec}(\mathbf{A})$	Column-order vectorization of the entries of $\mathbf{A}$.
$\mathrm{nnz}(\cdot)$	Number of non-zero entries of an object.

$\mathcal{T}$ is defined as the minimum number of rank-one tensors that generate $\mathcal{T}$ as their sum. We define the *CP* decomposition of rank R of a tensor $\mathcal{T}$ as the sum $\mathcal{T} = \sum_{r=1}^{R} u_r \circ v_r \circ w_r$, and we denote it by $\mathcal{T} = [\![U, V, W]\!]$, where U, V and W are matrices with R columns given by u_r, v_r, and w_r. An important operation in the context of tensors is the *mode-k product* between a tensor $\mathcal{T} \in \mathbb{R}^{N_1 \times N_2 \times N_3}$ and a matrix $U \in \mathbb{R}^{R \times N_k}$ for $k \in \{1, 2, 3\}$, which is denoted by $\mathcal{Y} = \mathcal{T} \times_k U \in \mathbb{R}^{N_1 \times \cdots \times R \times \cdots \times N_3}$ where the k-th dimension is replaced by R. In slice form, the mode-k product has matricized representation $\mathbf{Y}_{(k)} = U\mathbf{T}_{(k)}$. In components, it is represented as: $y_{i_1 \ldots i_{k-1} \alpha i_{k+1} \ldots i_3} = \sum_{i_k=1}^{N_k} t_{i_1 i_2 i_3} u_{\alpha, i_k}$.

2 Fast matrix multiplication

2.1 Evaluation of bilinear form using low-rank tensor decompositions

Let X, Y, Z be finite-dimensional vector spaces of dimensions I, J, K respectively. A *bilinear form* is a mapping $f : X \times Y \to Z$ that is linear in each entry, in other words such that $f(x, \cdot)$ and $f(\cdot, y)$ are linear for all $x \in X$ and $y \in Y$. Each component f_k is a bilinear form $f_k : X \times Y \to \mathbb{R}$, which can be represented by a matrix slice $\mathbf{T}_{(k)}$ of a tensor $\mathcal{T}$ such as

$$f_k(x, y) = x^\top \mathbf{T}_{(k)} y = \sum_{i=1}^{I} \sum_{j=1}^{J} t_{ijk} x_i y_j, \tag{1}$$

for all $k \in \{1, \ldots, K\}$. Hence we can construct a tensor $\mathcal{T}$ which represents f by taking the slices $\mathbf{T}_{(k)}$ that represent f_k. In more succint tensor notation,

$$f = \mathcal{T} \times_1 x^\top \times_2 y^\top. \tag{2}$$

Numerically evaluating the bilinear form in Equation 1 has a high computational cost driven by the multiplication that depends by the non-zero entries of $\mathcal{T}$. We call these multiplications *active* to emphasize their dominant computational role. We can reduce the cost of evaluating $f(x, y)$ by considering a CP decomposition $[\![U, V, W]\!]$ of rank R of $\mathcal{T}$. Denoting as u_l, v_l, w_l the columns of U, V and W, we have the decomposition $\mathcal{T} = \sum_{r=1}^{R} u_r \circ v_r \circ w_r$, and by performing a substitution into Equation 2

component-wise, we get

(3)
$$\begin{aligned} f_k(x,y) &= \left(\left(\sum_{r=1}^{R} u_r \circ v_r \circ w_r\right) \times_1 x^\top \times_2 y^\top\right)_k = \left(\sum_{r=1}^{R} (u_r \circ v_r \circ w_r) \times_1 x^\top \times_2 y^\top\right)_k \\ &= \sum_{i=1}^{I}\sum_{j=1}^{J}\sum_{r=1}^{R} u_{ir}v_{jr}w_{kr}x_iy_j = \sum_{r=1}^{R}\left(\sum_{i=1}^{I} u_{ir}x_i\right)\cdot\left(\sum_{j=1}^{J} v_{jr}y_j\right) w_{kr} \\ &= \sum_{r=1}^{R} (u_r^\top x)\cdot(v_r^\top y) w_{kr}, \end{aligned}$$

for every $k = 1, \dots, K$. Since this holds for every k, we can write

(4)
$$f(x,y) = \sum_{l=1}^{R} (u_l^\top x)\cdot(v_l^\top y) w_l.$$

Here we highlight with $(\cdot)$ the active multiplications. Now, evaluating f requires exactly R active multiplications (and $O(R)$ additions).

2.2 Recursive matrix multiplication

Recall that matrix multiplication for two matrices $A \in \mathbb{R}^{M\times N}$ and $B \in \mathbb{R}^{N\times P}$ is defined as

$$C = AB = \left(\sum_{k=1}^{N} A_{ik}B_{kj}\right)_{\substack{1\le i\le M,\\ 1\le j\le P}} = \left[A_{:1}|\cdots|A_{:N}\right]\begin{bmatrix} B_{1:} \\ \vdots \\ B_{N:}\end{bmatrix} = \sum_{k=1}^{N} A_{:k}B_{k:}.$$

From this expression easily follows the bilinearity of the map $(A, B) \mapsto AB$, and we indicate with $\langle M, N, P\rangle$ the form that represents the multiplication between A and B. By this property, we can rewrite Equation 2 putting $x = \mathrm{vec}(A)$, $y = \mathrm{vec}(B)$ and the output vector $z = \mathrm{vec}(C^\top)$. This fixes the natural ordering on the entries of $\mathrm{vec}(A)$ and $\mathrm{vec}(B)$ and the inverse ordering on $\mathrm{vec}(C^\top)$. This yields a representation of the tensor $\mathcal{T} \in \mathbb{R}^{MN\times NP\times MP}$ of matrix multiplication $\langle M, N, P\rangle$ from Equation 2 such that

(5)
$$\mathrm{vec}(C^\top) = \mathcal{T} \times_1 \mathrm{vec}(A)^\top \times_2 \mathrm{vec}(B)^\top.$$

If we are given a CP decomposition of rank r of the tensor $\mathcal{T} = [\![U, V, W]\!]$, by Equation 1 this expression becomes

(6)
$$Vec(C^\top) = \sum_{l=1}^{R} (\underbrace{u_l^\top \mathrm{vec}(A)}_{s_l})\cdot(\underbrace{v_l^\top \mathrm{vec}(B)}_{t_l}) w_l = \sum_{l=1}^{R} (\underbrace{s_l \cdot t_l}_{m_l}) w_l,$$

The expressions above allow us to define fast matrix multiplications algorithms for any base case $\langle m, n, p\rangle$.

2.3 The classic matrix multiplication algorithm tensor formulation

We present the $\langle 2, 2, 2\rangle$ classical matrix multiplication algorithm in the context of tensors. When $M, N, P > 2$ we can always partition them in blocks as:

$$(7) \qquad \begin{matrix} & \overbrace{\phantom{A_{11}}}^{\lfloor N/2 \rfloor} & \overbrace{\phantom{A_{12}}}^{\lceil N/2 \rceil} \\ \lfloor M/2 \rfloor \{ & A_{11} & A_{12} \\ \lceil M/2 \rceil \{ & A_{21} & A_{22} \end{matrix}, \qquad \begin{matrix} & \overbrace{\phantom{B_{11}}}^{\lfloor P/2 \rfloor} & \overbrace{\phantom{B_{12}}}^{\lceil P/2 \rceil} \\ \lfloor N/2 \rfloor \{ & B_{11} & B_{12} \\ \lceil N/2 \rceil \{ & B_{21} & B_{22} \end{matrix},$$

where the sizes of the subblocks are depicted in the above equation. This algorithmic formulation is often called *divide-and-conquer*, as it consists in breaking the problem into smaller subproblems, and later recombining the results.

Remark 2.1. While this floor/ceiling partition is mathematically valid for classical block matrix multiplication, it cannot be directly applied to Strassen's algorithm or other CP-based fast multiplication schemes if dimensions are odd. This is because Strassen's algorithm performs additions between subblocks (e.g., $A_{11} + A_{22}$), which are only defined if the subblocks have identical dimensions. Thus, in practice, odd or non-power-of-two dimensions are first handled using dynamic peeling (Section 4.1) to obtain an even-dimensioned core, which is then partitioned into equal-sized blocks.

In particular, we can view the product AB as the product between 2×2 block matrices

$$(8) \qquad \begin{bmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{bmatrix} = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix} \cdot \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix} = \begin{bmatrix} A_{11}B_{11} + A_{12}B_{21} & A_{11}B_{12} + A_{12}B_{22} \\ A_{21}B_{11} + A_{22}B_{21} & A_{21}B_{12} + A_{22}B_{22} \end{bmatrix}.$$

The naive algorithm proceeds by combining a set of eight matrix multiplications with four additions:

$$\begin{aligned} M_1 &= A_{11} \cdot B_{11} & M_2 &= A_{12} \cdot B_{21} & M_3 &= A_{11} \cdot B_{12} \\ M_4 &= A_{12} \cdot B_{22} & M_5 &= A_{21} \cdot B_{11} & M_6 &= A_{22} \cdot B_{21} \\ M_7 &= A_{21} \cdot B_{12} & M_8 &= A_{22} \cdot B_{22} \end{aligned}$$

$$\begin{aligned} C_{11} &= M_1 + M_2 & C_{12} &= M_3 + M_4 \\ C_{21} &= M_5 + M_6 & C_{22} &= M_7 + M_8 \end{aligned}$$

The number of flops performed by standard matrix multiplication for $N \times N$ matrices is $T(N) = 8T(\frac{N}{2}) + 4(\frac{N}{2})^2$ for $N > 1$. This can be solved by the master method to get $T(N) = 2N^3 - N^2$, where we assume that N is a power of two. In Section 4.1 we explain how to handle all dimensions.

For example, when $m = n = p = 2$, we get the tensor with frontal slices

$$\mathbf{X}_1 = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}, \quad \mathbf{X}_2 = \begin{bmatrix} 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \end{bmatrix}, \quad \mathbf{X}_3 = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix}, \quad \mathbf{X}_4 = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}.$$

This yields, for example,

$$X_3 \times_1 \mathrm{vec}(A)^\top \times_2 \mathrm{vec}(B)^\top = \begin{bmatrix} a_{11} & a_{21} & a_{12} & a_{22} \end{bmatrix} \begin{bmatrix} 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix} \begin{bmatrix} b_{11} \\ b_{21} \\ b_{12} \\ b_{22} \end{bmatrix} = a_{21}b_{11} + a_{22}b_{21} = c_{21}.$$

Note that the formula in Equation 6 in the context of block matrix multiplication in Equation 8 yields to

$$\mathrm{vec}(A) = \begin{bmatrix} A_{11} \\ A_{21} \\ A_{12} \\ A_{22} \end{bmatrix}, \quad \mathrm{vec}(B) = \begin{bmatrix} B_{11} \\ B_{21} \\ B_{12} \\ B_{22} \end{bmatrix}, \quad \mathrm{vec}(C^\top) = \begin{bmatrix} C_{11} \\ C_{12} \\ C_{21} \\ C_{22} \end{bmatrix}.$$

Also the factors s_l and t_l in Equation 6 become

$$s_l = u_l^\top \mathrm{vec}(A) = \sum_{i=1}^{4} U_{li} \mathrm{vec}(A)_i, \quad t_l = V_l^\top \mathrm{vec}(B) = \sum_{i=1}^{4} V_{li} \mathrm{vec}(B)_i.$$

2.4 Strassen algorithm using CP

The best-known fast algorithm is due to Strassen, and follows the same $\langle 2, 2, 2 \rangle$ block structure:

$$\begin{aligned}
&S_1 = A_{11} + A_{22} && S_2 = A_{21} + A_{22} && S_3 = A_{11} \\
&S_4 = A_{22} && S_5 = A_{11} + A_{12} && S_6 = A_{21} - A_{11} \\
&S_7 = A_{12} - A_{22} \\
&T_1 = B_{11} + B_{22} && T_2 = B_{11} && T_3 = B_{12} - B_{22} \\
&T_4 = B_{21} - B_{11} && T_5 = B_{22} && T_6 = B_{11} + B_{12} \\
&T_7 = B_{21} + B_{22}
\end{aligned}$$

$$\begin{aligned}
&M_r = S_r T_r, \quad 1 \leq r \leq 7 \\
&C_{11} = M_1 + M_4 - M_5 + M_7 && C_{12} = M_3 + M_5 \\
&C_{21} = M_2 + M_4 && C_{22} = M_1 - M_2 + M_3 + M_6
\end{aligned}$$

This algorithm uses 7 matrix multiplications and 18 matrix additions. The number of flops performed by the algorithm is $T(N) = 7T(\frac{N}{2}) + 18(\frac{N}{2})^2$ for $N > 1$, solving the recursion the complexity is $T(N) = 7N^{\log_2 7} - 6N^2 = O(N^{\log_2 7}) = O(N^{2.81})$. This algorithm corresponds to a low-rank tensor decomposition represented by the following triplet of matrices, each with 7 columns:

$$U = \begin{bmatrix} 1 & 0 & 1 & 0 & 1 & -1 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 1 \\ 0 & 1 & 0 & 0 & 0 & 1 & 0 \\ 1 & 1 & 0 & 1 & 0 & 0 & -1 \end{bmatrix},$$

$$V = \begin{bmatrix} 1 & 1 & 0 & -1 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 1 \\ 1 & 0 & -1 & 0 & 1 & 0 & 1 \end{bmatrix},$$

$$W = \begin{bmatrix} 1 & 0 & 0 & 1 & -1 & 0 & 1 \\ 0 & 1 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 1 & 0 & 0 \\ 1 & -1 & 1 & 1 & 0 & 0 & 0 \end{bmatrix}.$$

While we cannot use less than 7 multiplications for the $\langle 2, 2, 2 \rangle$ algorithm [4], there are natural extensions to Strassen algorithm:

1. We can reduce the number of additions from 18 down to 15, which leads to the Strassen-Winograd algorithm [4].
2. We can change constant on the leading term by choosing a bigger base case dimension (and using the standard algorithm (also called *gemm*) on the base case).
3. We can use blocking schemes different from $\langle 2, 2, 2 \rangle$.

We implement this extension in practice using the Rust programming language, targeting exact algorithms arising from the CP decomposition matrices made freely available by Benson and Ballard [1]. Table 2 shows a summary of fast algorithms for different base cases.

3 Project overview

3.1 Fork overview and comparison with C++ version

The implementation consists of a Rust port of the c++ project made by Benson and Ballard summarized in [1], available in the relative repository under BSD 2-Clause. We will refer to this project as "the C++ project" or "Benson and Ballard's project". They provide a fast and efficient framework for benchmarking parallel and sequential fast matrix multiplication algorithms, achieved through code generation, using lapack and MKL dgemm for numerical computation, and OpenMP for parallelization. We develop a Rust project that aims to replicate a parallel framework for fast multiplication algorithms, but with structural differences and current limitations. The main similarities are that both use MKL for base, but the Rust project also uses Faer. In the C++ project, MKL is imported directly, while in Rust it is wrapped via FFI. Besides tool choices, the C++ algorithms are code-generated by a script that parses CP tensor decomposition slices and translates the matrix multiplication expression into C++ code, while Rust loads the tensor at runtime [1] and then applies the pseudocode in Algorithm 1. The advantage of the first is

[1] when program is launched, before doing the multiplication, which does not affect speed since the matrices decomposition is cached at runtime

Table 2: Summary of fast algorithms. Algorithms without citation were found by the authors of [1]. The number of multiplications is equal to the rank R of the corresponding tensor decomposition. The multiplication speedup per recursive step is the expected speedup if matrix additions were free. Note that this speedup does not determine the fastest algorithm because the maximum number of recursive steps depends on the size of the subproblems created by the algorithm.

Algorithm Base Case $\langle m, n, p \rangle$	Multiplications (Fast, R)	Multiplications (Classical, mnp)	Speedup per Recursive Step
$\langle 2, 2, 3 \rangle$	11	12	9%
$\langle 2, 2, 5 \rangle$	18	20	11%
$\langle 2, 2, 2 \rangle$ [5]	7	8	14%
$\langle 2, 2, 4 \rangle$	14	16	14%
$\langle 3, 3, 3 \rangle$	23	26	17%
$\langle 2, 3, 3 \rangle$	15	18	20%
$\langle 2, 3, 4 \rangle$	20	24	20%
$\langle 2, 4, 4 \rangle$	26	32	23%
$\langle 3, 3, 4 \rangle$	29	36	24%
$\langle 3, 4, 4 \rangle$	38	48	26%
$\langle 3, 3, 6 \rangle$ [6]	40	54	35%

speed, at the cost of more complex code and parallelism. The advantage of the second is simplicity of code, at the cost of speed (at the current state, but which it could be improved by adding optimization, we will address these issues in Section 6). The C++ project can handle multiplications between rectangular matrices, as well as APA algorithms, while ours does not. Also, while it could work, we have not tested subexpression-eliminated algorithms.

3.2 Algorithm schema

We now discuss the algorithm schema for Strassen algorithm for clarity, though results generalize easily for any $\langle m, n, p \rangle$ base case. Algorithm 1 shows our implementation of Strassen's algorithm, which easily extends to other CP algorithms by changing base conditions and block sizes—this is how it is implemented in practice.

3.3 Execution platform and reproducibility

Platform

Since the comparison targets both sequential and parallel matrix multiplication kernels, we ran all experiments on one node of a dual-socket AMD EPYC 7763 server (64 physical cores per socket, 2 threads per core) with 2 TB of RAM. The system ran Ubuntu 24.04.5 LTS with Linux 5.15.0-179-generic on x86_64. For sequential benchmarks, library-level threading was restricted with `OMP_NUM_THREADS=1` and `MKL_NUM_THREADS=1`. These choices remove scheduler migration and library-level parallelism from the measurements to isolate microarchitectural performance and compiler optimizations without the interference of thread scheduling and synchronization overheads.

Algorithm 1 Strassen algorithm using CP factorization.

procedure STRASSEN(A, B, N)
  if STOP-RECURSING is True **then**
    return $A \cdot B$
  end if
  if $N = 2$ **then**
    STRASSEN-BASE-MATMUL(A, B)
  end if
  if $N \bmod 2 > 0$ **then**
    DYNAMIC-PEELING(A, B, N) ▷ Recursive call on peeled blocks.
  end if
  Compute $\mathrm{vec}(A)$ and $\mathrm{vec}(B)$ with block size $\frac{N}{2}$.
  for $r = 1, \ldots, R$ **do**
    $S_r \leftarrow \sum_{i=1}^{4} U_{i,r} \, \mathrm{vec}(A)_i$
    $T_r \leftarrow \sum_{i=1}^{4} V_{i,r} \, \mathrm{vec}(B)_i$
    $M_r \leftarrow$ STRASSEN($S_r, T_r, \frac{N}{2}$) ▷ Recursive call.
  end for
  $\mathrm{vec}(C^\top) = \sum_{r=1}^{r} M_r w_r$
  return C.
end procedure

Environment

The software environment is configured and reproduced using Environment Modules. The configuration loads `gcc/13.2.0` as the base compiler toolchain, alongside the `intel-oneapi-compilers` version `2025.0.4-gcc-11.4.0`. To provide an optimized BLAS/LAPACK backend, the environment loads `intel-oneapi-mkl`. Although this combination introduces minor version discrepancies, it was required to prevent compilation and linking failures under the available cluster toolchain.

Compilation

When building the C wrappers and Rust bindings, all compilation targets are explicitly optimized for the server target architecture. In the C++ project, to support both sequential and parallel modes within a single executable, the MKL backend was dynamically linked against the LLVM OpenMP threading layer passing `-liomp5`. We build the Rust code with the nightly-2026-06-23 toolchain because this release uses LLVM 22, which generates optimized vector instructions leveraging AVX2 and FMA. Since the C/C++ wrappers and Rust code compile with matching CPU targets, a custom `build.rs` script compiles the Intel MKL FFI C wrapper (`mkl.c`) using `gcc`. During linking, `build.rs` ensures that the Intel MKL dynamic libraries are correctly loaded. To eliminate the need for manual `LD_LIBRARY_PATH` configuration at runtime, the MKL library path is compiled into the executable binaries as a runtime.

4 Practical optimizations

Fast matrix multiplication algorithms such as Strassen face two significant problems in practical implementation: recursion cutoff and input matrix sizes not matching the base case dimensions. In the parallel

case, there are also scheduling problems to consider to avoid algorithmic overhead.

4.1 Dynamic Peeling

In fast matrix multiplication algorithms we want to multiply $M \times N \times P$ matrices by splitting them into blocks as in Equation 7. If M, N and P are not divisible by the base case $\langle m, n, p \rangle$, then the matrices cannot be partitioned into equal blocks. The easiest fix for this is padding the matrix with zeros until a divisible size is reached, but this incurs significant memory overhead. A more efficient approach, called *dynamic peeling*, consists in stripping the extra row off and/or column and adding their contribution to the final results.

We present dynamic peeling for Strassen's algorithm for the sake of clarity—the construction easily adapts to other base cases. For the matrix multiplication $C = AB$, where A is an $M \times N$ matrix and B is an $N \times P$ matrix, the matrix A is split into an $(M-1) \times (N-1)$ core matrix A_{11}, alongside its peeled boundary vectors a_{12}, a_{21}, and the scalar corner element a_{22}. Similarly, the matrix B is split into an $(N-1) \times (P-1)$ core matrix B_{11}, with its corresponding peeled components b_{12}, b_{21}, and b_{22}:

$$A = \left[\begin{array}{c|c} A_{11} & a_{12} \\ \hline a_{21} & a_{22} \end{array}\right] \begin{array}{l} \} M-1 \\ \} 1 \end{array}, \qquad B = \left[\begin{array}{c|c} B_{11} & b_{12} \\ \hline b_{21} & b_{22} \end{array}\right] \begin{array}{l} \} N-1 \\ \} 1 \end{array}$$

The final block matrix product is computed directly through a combination of the core Strassen multiplication ($A_{11}B_{11}$) and low-rank vector/scalar fixup modifications:

$$\left[\begin{array}{c|c} C_{11} & c_{12} \\ \hline c_{21} & c_{22} \end{array}\right] \begin{array}{l} \} M-1 \\ \} 1 \end{array} = \left[\begin{array}{c|c} A_{11}B_{11} + a_{12}b_{21} & A_{11}b_{12} + a_{12}b_{22} \\ \hline a_{21}B_{11} + a_{22}b_{21} & a_{21}b_{12} + a_{22}b_{22} \end{array}\right]$$

Here, only the sub-product $A_{11}B_{11}$ is executed recursively using Strassen's matrix multiplication, while all other terms are computed via standard matrix multiplication.

Although the mathematical formulation suggests computing multiple small matrix additions and products, such a block-wise approach would suffer from high library call overhead and poor cache locality. In our Rust implementation, we consolidate the boundary corrections into larger, contiguous GEMM operations: the right columns are computed via a single product AB_{extra} (where B_{extra} represents the peeled columns of B), and the bottom rows are computed via $A_{extra}B_{core}$ (where A_{extra} represents the peeled rows of A). This consolidation maintains high arithmetic intensity and maximizes vendor BLAS efficiency.

4.2 Parallel algorithms for shared memory

Matrix multiplication is a problem subject to both computational and memory bandwidth bounds. The first is limited by the amount of computation the device can handle (i.e., cores, CPU/GPU speed), while

the latter is limited by memory transfer throughput. In a parallel implementation, when dealing with recursive algorithms which use divide-and-conquer (like Strassen), it is important in practice to balance algorithmic load between threads to avoid overhead caused by both taking recursive branches and doing parallel matrix multiplication. To address this issue, we adopt the scheduling scheme suggested by Benson and Ballard in their work [1]. We use Rust's Rayon to handle parallelization, and we can summarize the parallel scheduling as following:

- **BFS:** Each recursive matrix multiplication routine and the associated matrix additions are launched by Rayon as an iterable, and then are collected in an array of matrices $[M_1, \ldots, M_r]$.
- **DFS:** Each vendor matrix multiplication call uses all threads and the recursion tree is evaluated sequentially.
- **Hybrid:** The Hybrid approach developed in [1] compensates for the load imbalance in BFS by applying DFS on a subset of the base case problems. With L levels of recursion and P threads, the hybrid algorithm applies task parallelism (BFS) to the first $R^L - (R^L \bmod P)$ multiplications. Because the number of subproblems is a multiple of P, this part is load balanced. On the remaining $R^L \bmod P$ subproblems, all threads are used on each matrix multiplication (DFS).

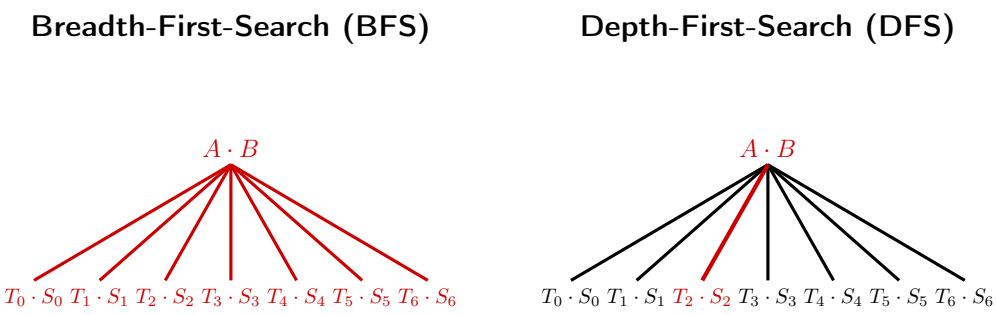


Figure 1: BFS and DFS load balancing. The parallelized tasks are in red.

4.3 Recursive cutoff strategies

In divide-and-conquer algorithms such as Strassen, we cannot afford taking many recursive steps, as spawning many children constitutes a significant recursive overhead, especially for larger matrices. Thus, finding an optimal cutoff is important for stopping the recursion and switch to standard GEMM. We try two techniques: the first is breaking by size, that is when the sizes of matrices in the recursion fall under a certain size, the latter is stopping after a fixed number L of recursive levels.

The authors in [1] explain a heuristic process for finding the optimal cutoff size, which can be resumed in looking at the performance plot of the GEMM function, where (figure) the function exibits a ramp-up and then flatten for large dimensions. They take a recursion step only if the recursive sub-problem fall in the flat part of the curve. This is done manually by the authors. During the experimentation we developed a function to plot this curve, and its derivative (using splines) to automatically find this size. Due to the performance characteristics of our current implementation, this automated cutoff selection was not integrated into the final benchmark runs, and a fixed cutoff size was used instead.

5 Experimental results

5.1 Metrics

All benchmarks are recorded using Rust's system clock, performing warm-up runs before measuring each matrix multiplication. In order to compare the performance of matrix multiplication algorithms with different computational costs, we use the *effective* GFLOPS metric for $M \times N \times P$ matrix multiplication:

$$\text{effective GFLOPS} = \frac{2MNP - MP}{\text{time in seconds} \cdot 10^9}, \tag{9}$$

where the numerator can be rewritten as $2MNP - MP = MNP + M(N-1)P$, which is the number of multiplications and additions performed for classical matrix multiplication. This allows us to compare all of the algorithms on an inverse-time scale, normalized by problem size [1]. For fast algorithms, considering both multiplications and additions in the metric is more accurate than taking multiplications only [7].

5.2 Measuring impact of base GEMM

We aim to understand how the choice of the base matrix multiplication impacts the performance of fast algorithms. In our experiments, we use random square matrices with double-precision (64-bit) floating-point entries in $(-1, 1)$ of sizes $N \in \{2, 4, \ldots, 2^k\}$. As the standard matrix multiplication algorithm implemented in GEMM costs $O(N^3)$ while Strassen costs $O(N^{2.81})$, we expect the fast algorithm to outperform standard GEMM for sufficiently large N. We thus seek to determine the crossover dimension N. Such N cannot be determined a priori, as it depends on hardware features (such as cache capacities, SIMD widths) and software platform implementation details. In other words, we expect fast algorithms to surpass the base GEMM curves shown in Figure 2.

5.3 Comparison of MKL dgemm FFI and Faer matmul

In this work, we consider two different matrix multiplication routines: MKL dgemm and faer base matmul.

- **Faer matmul:** The method is entirely written in Rust and is part of the faer library, which is designed as a lightweight, performant matrix multiplication kernel with fewer microkernel optimizations than vendor GEMMs. See [8] for a method overview from the library's author.
- **MKL dgemm:** is a routine in the Intel Math Kernel Library (MKL) for multiplying double-precision matrices. In Rust, this routine runs via a Foreign Function Interface (FFI) calling the C wrapper `cblas_dgemm`. In the C++ project, the function is called directly. Both projects use the same library version on the architecture.

From Figure 2, we see that all functions have a ramp-up phase and then flatten for larger sizes as they transition from memory-bound overheads to compute-bound SIMD execution. In the sequential case, both libraries plateau beyond $N = 4096$, with MKL dgemm achieving approximately 48.0 GFLOPS at $N = 65536$ and faer achieving 43.1 GFLOPS; this gap is attributed to the highly optimized assembly

microkernels embedded in MKL. In the parallel case (run on the AMD EPYC 7763 GPU partition using 16 threads), there is a noticeable discrepancy between parallel MKL dgemm called directly from C++ and that invoked via Rust FFI. We believe this result is run-dependent, as we observed in some experimental runs that the FFI version was slightly faster than the native C++ MKL, likely due to OS scheduling and thread pool fluctuations on the shared cluster nodes. This discrepancy disappears at $N = 65536$, where both achieve 16.16 GFLOPS/core. Additionally, parallel faer performs best at smaller dimensions ($N < 8192$), achieving 14.18 GFLOPS/core at $N = 2048$ (compared to MKL's 3.88 GFLOPS/core) and peaking at 17.67 GFLOPS/core at $N = 4096$, outperforming MKL due to its lightweight scheduling layer. However, at $N \geq 8192$ faer is surpassed by MKL dgemm, which reaches 18.58 GFLOPS/core at $N = 8192$ and sustains 16.16 GFLOPS/core at $N = 65536$ compared to faer's 13.65 GFLOPS/core.

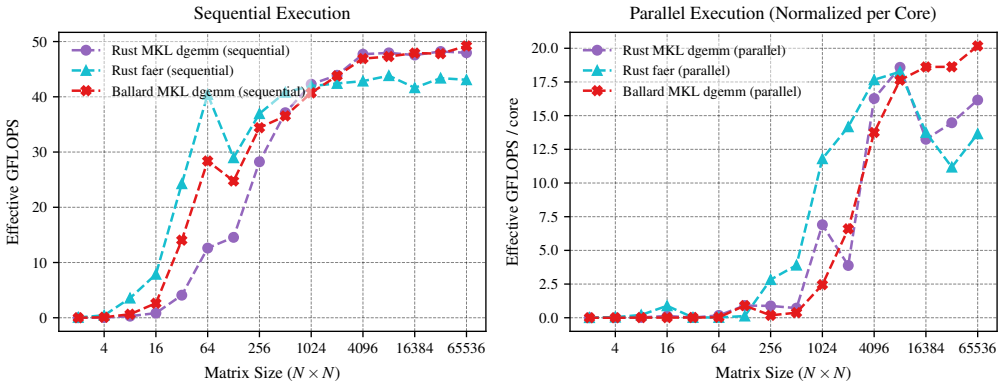


Figure 2: Comparison between MKL and faer in sequential and parallel execution.

5.4 Comparison of the level and size recursive cutoff strategies

The experimental results for sequential and parallel cutoff strategies are shown in Figures 3a and 3b, respectively. Figures 4a and 4b show the comparison of Strassen and base GEMM for the parallel case. In general, we notice that while the base GEMM performance differs between MKL and faer, the respective Strassen variants tend to show similar performance characteristics, as Figures 3a and 3b demonstrate.

In the sequential case (Figure 3a), sequential Strassen starts to outperform standard GEMM around $N = 8192$. The performance gains become more pronounced at larger dimensions: at $N = 65536$, sequential Strassen achieves approximately 61.2 GFLOPS ($L = 2$) for the MKL base and 54.8 GFLOPS ($L = 2$) for the faer base, outperforming their respective base GEMM counterparts (48.0 GFLOPS and 43.1 GFLOPS) by approximately 27.5% and 27.1% in throughput. The optimal cutoff strategy is a fixed size cutoff of 2048 or a recursion depth of $L = 2$, which balances Strassen's arithmetic advantage against the overhead of memory allocation.

In the parallel case (Figure 3b), the crossover point is shifted at $N = 16384$. At this dimension, the parallel BFS Strassen variant backed by MKL dgemm is still close to MKL dgemm, but at the maximum tested dimension, parallel BFS Strassen with $L = 3$ recursion levels achieves peak performance of approximately 28.96 GFLOPS/core, representing a 79.2% throughput increase over the parallel MKL base of 16.16 GFLOPS/core. Similarly, BFS Strassen backed by faer at $L = 3$ achieves 24.93 GFLOPS/core, outperforming the base (13.65 GFLOPS/core) by 82.6%. The parallel OpenMP execution inside the MKL threading layer and Rayon's dynamic scheduling in the BFS and Hybrid variants are critical to sustaining these gains at larger dimensions.

To summarize, best performance is obtained when less with $L = 2$ or when using an optimal cutoff (2048 for sequential, and 16384 for parallel), which is when less recursive steps are taken. Furthermore, an accurate implementation could include a dynamic switch between the two modes.

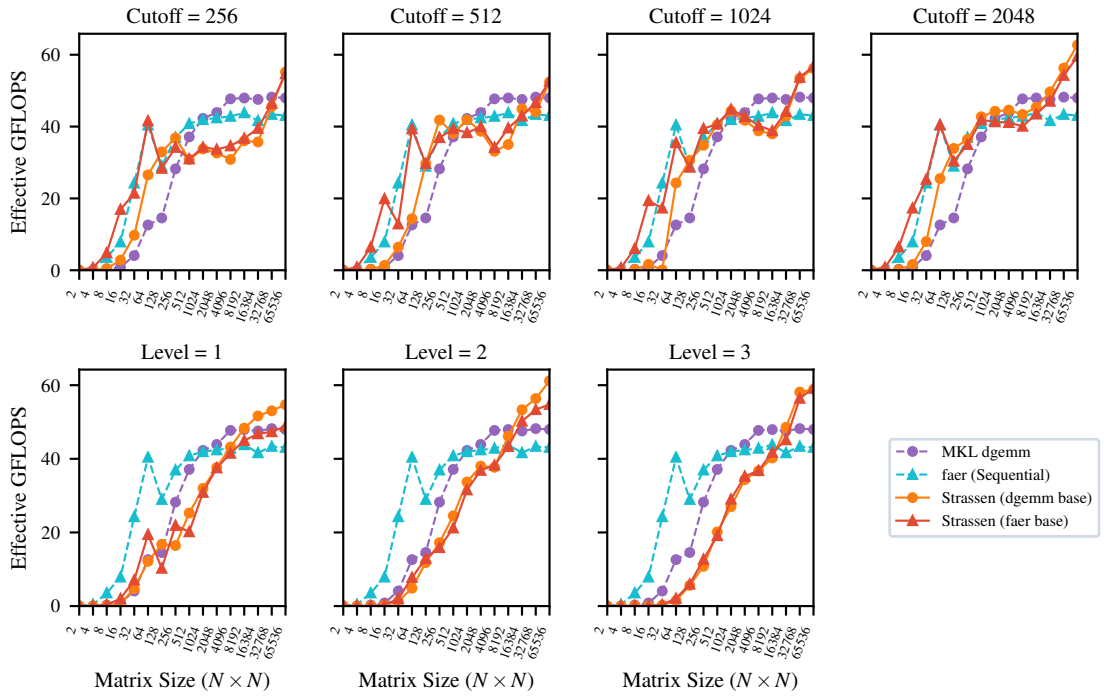
5.5 Finding the optimal cutoff for the parallel case

Figure 5 shows different recursive size thresholds for the parallel case. We observe that BFS consistently outperforms DFS across all cutoff configurations, showing also less sensitivity to the base case than DFS. Recall that in DFS scheduling paradigm the recursion tree is evaluated sequentially, parallelizing only the leaf-node GEMM operations. Consequently, DFS suffers from thread under-utilization during the recursive splits and matrix addition phases. To maximize DFS performance, the leaf-node GEMM size must be large enough to saturate the 16 cores; hence, DFS achieves its best performance at larger cutoffs. Conversely, BFS parallelizes the 7 subproblems at the top levels of the recursion tree using Rayon. With 16 available threads, a single level of recursion (7 tasks) leaves at least 9 threads idle, whereas going to 2 or 3 levels easily saturates the thread pool (for instance, when $L = 2$ there are $7^2 = 49$ tasks). However, choosing a cutoff that is too small (e.g., Cutoff 256 or 512) triggers an exponential growth in auxiliary memory allocations and memory bus contention, which degrades performance. For $N = 32768$ backed by MKL dgemm, BFS achieves its peak performance at Cutoff 2048 (19.76 GFLOPS/core), while for $N = 65536$ it peaks at Cutoff 4096 (29.41 GFLOPS/core). For the faer backend, the optimal cutoff is larger (Cutoff 8192 for $N = 32768$ achieving 17.24 GFLOPS/core, and Cutoff 4096 for $N = 65536$ achieving 25.91 GFLOPS/core). Hence due to inner parallelism efficiency of MKL and faer, MKL Strassen variants perform better when breaking smaller blocks, while faer making it beneficial to transition to the base case sooner thank faer.

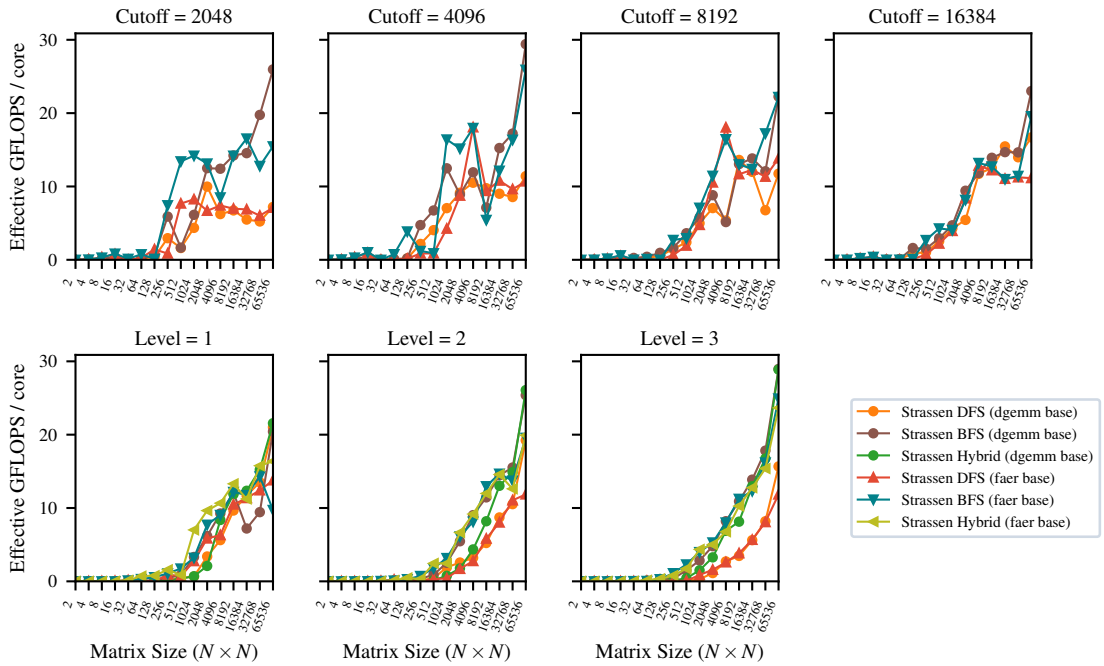
5.6 Numerical comparison with Benson&Ballard Strassen

We perform a numerical performance comparison between our optimized Rust implementation of Strassen's algorithm and the reference C++ implementation by Benson and Ballard [1]. The results are shown in Figure 6. We compare the algorithms across one, two, and three levels of recursion using different thread scheduling paradigms from Section 4.2. We choose only to compare cutoff by level as is currently the only supported by the C++ project.

We observe that our implementation is highly competitive in the sequential case, with the dgemm-backed version yielding the highest overall throughput. In the parallel DFS case, Ballard's C++ code outperforms our algorithm (e.g., at $N = 65536$ with $L = 2$, Ballard DFS achieves 27.27 GFLOPS/core compared to our Rust DFS's 19.23 GFLOPS/core). This discrepancy is because Ballard's implementation parallelizes the recursive matrix additions across threads using OpenMP, whereas our Rust DFS performs the additions and splits sequentially on the main thread, parallelizing only the leaf GEMM operations. On a 16-thread machine, these sequential matrix additions introduce a severe memory-bound bottleneck. However, in the parallel BFS and Hybrid scheduling paradigms, our Rust implementation significantly outperforms the C++ reference at large scales. At $N = 65536$ with $L = 3$, our Rust BFS achieves 28.96 GFLOPS/core (outperforming Ballard BFS's 26.32 GFLOPS/core by 10.0% in throughput), and our Rust Hybrid achieves 28.88 GFLOPS/core (when Ballard Hybrid's scores 21.74 GFLOPS/core by 32.8% in throughput). Crucially, while Ballard's C++ Hybrid strategy suffers from performance degradation at $L = 3$, our Rust Hybrid implementation maintains flat, high scaling. This confirms that Rayon's dynamic scheduler in Rust manages the recursive task scheduling and load-balancing with better efficiency compared to OpenMP's runtime configuration.

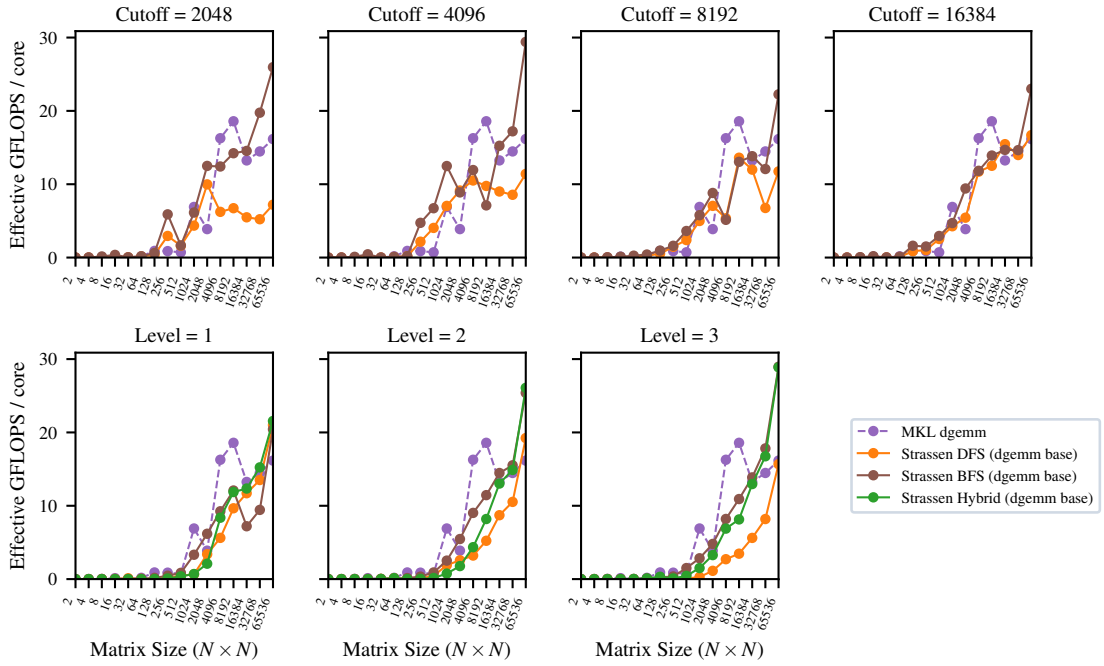


(a) Sequential execution.

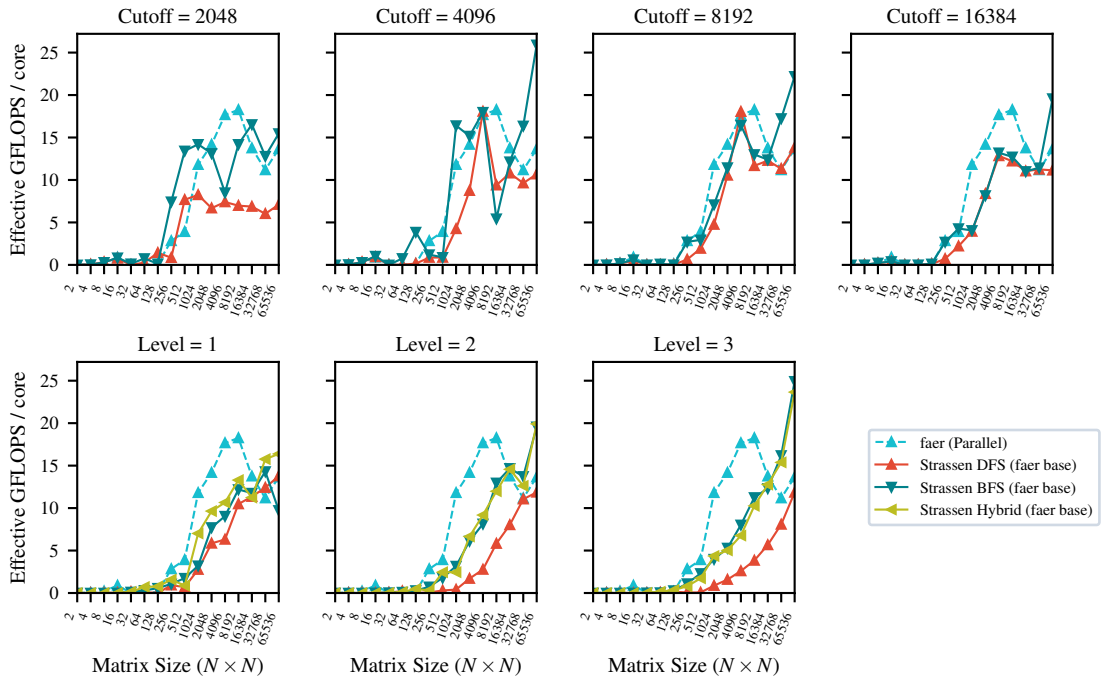


(b) Parallel execution (normalized per core).

Figure 3: Performance comparison of Strassen variants (DFS, BFS, Hybrid) across recursion levels and size cutoffs in sequential and parallel executions.

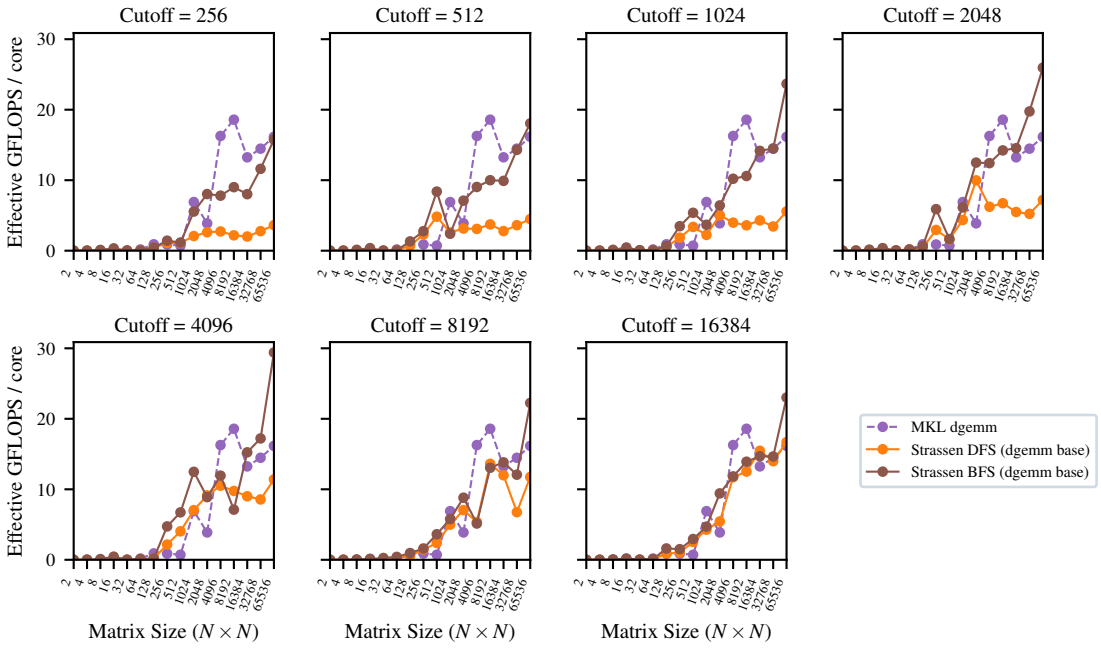


(a) Parallel dgemm base.

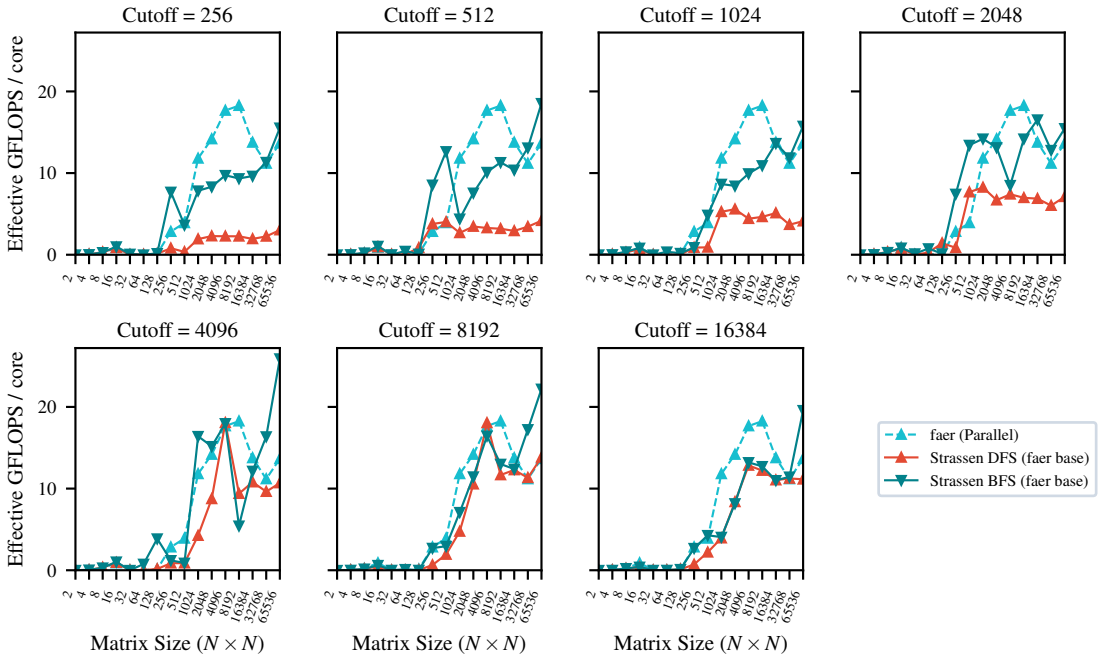


(b) Parallel faer base.

Figure 4: Parallel performance comparison (normalized per core) of Strassen variants (DFS, BFS, Hybrid) against parallel GEMM, across cutoff sizes (first row), and recursion levels (second row).



(a) Parallel dgemm base.



(b) Parallel faer base.

Figure 5: Parallel matrix multiplication performance comparison (normalized per core) of Strassen variants (DFS, BFS, Hybrid) against parallel GEMM across size cutoffs.

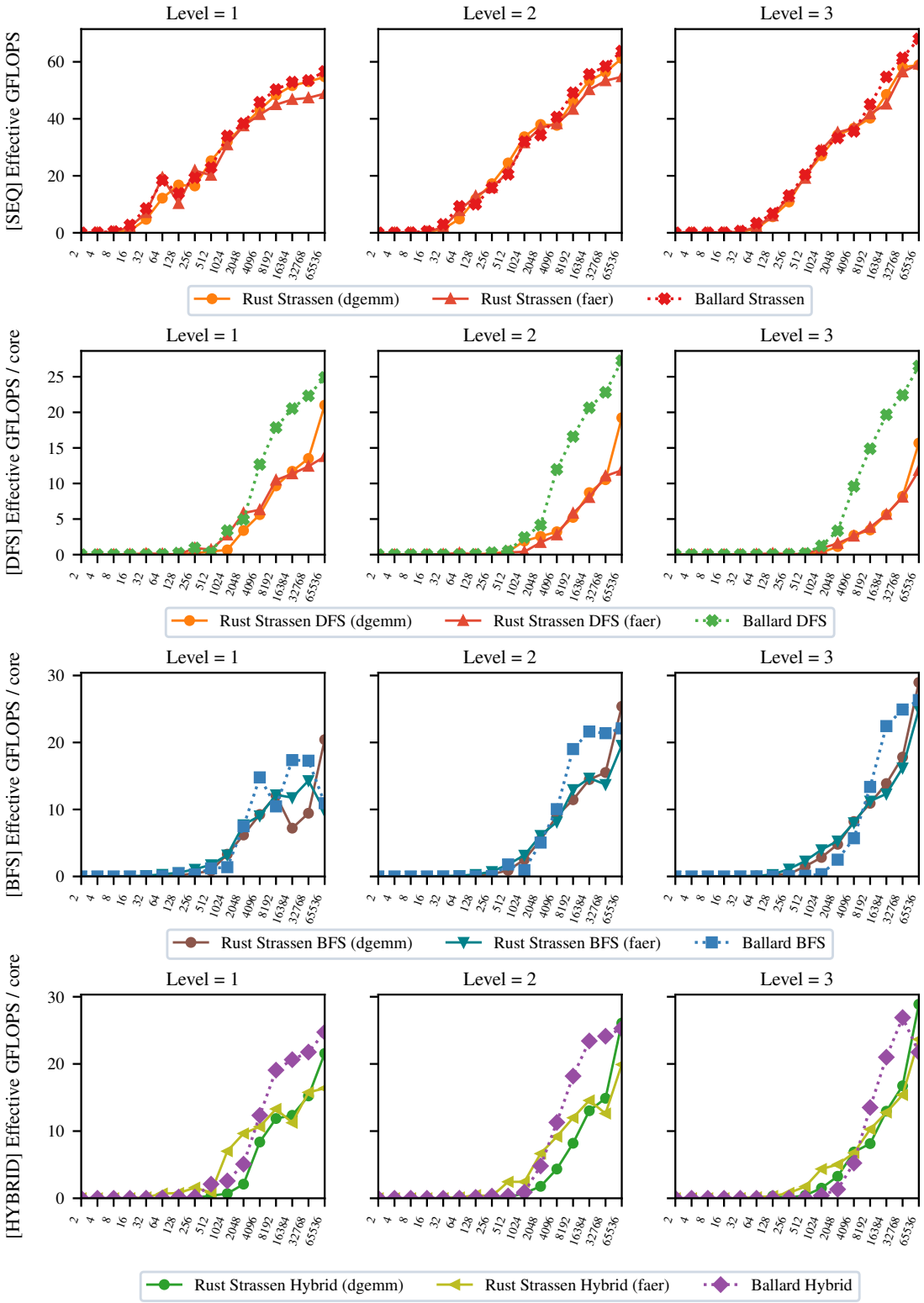


Figure 6: Performance comparison of our Rust Strassen implementation (using dgemm and faer base cases) against the Benson and Ballard reference benchmarks, sequential and parallel execution.

6 Conclusion & Further work

In conclusion, we noticed fast matrix multiplication algorithm show more optimal performance than the base GEMM libraries both in sequential and parallel cases, even with little performance optimizations. We believe that results of our work can be further improved by pursuing some of the following steps:

- **Profiling and SIMD:** We believe that the code can be made more efficient by doing a finer refactoring informed by profiling results. Many algorithm steps can be optimized by introducing more SIMD operations.
- **GEMM comparison:** Though we choose mkl-dgemm for a better comparison, our ffi bindings easily extend to any other GEMM, such as OpenBlas, which we leave to further work.
- **Gpu/distributed version:** We would like to port the project to GPU or distributed computing to extend results with higher parallelism and test larger sizes.
- **Run on Xeon:** The project could be run on high performance Intel Xeon nodes on cluster, which might show impact of Intel MKL dgemm proprietary optimizations.

7 Work reproducibility

All the code is available at the repository fast-matmul.

8 Acknowledgments

We acknowledge the authors of [2] for providing instructions for linking MKL with Rust, and the department of Mathematics of the University of Pisa for offering access to their HPC cluster.

References

[1] Austin R. Benson and Grey Ballard. A framework for practical parallel fast matrix multiplication. In *Proceedings of the 20th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming*, PPoPP 2015, page 42–53, New York, NY, USA, 2015. Association for Computing Machinery.

[2] Luca Lombardo and Fabio Durastante. Evaluating rust for sparse matrix kernels in scientific computing, 2026.

[3] Grey Ballard and Tamara G. Kolda. *Tensor Decompositions for Data Science*. Cambridge University Press, 2025.

[4] S. Winograd. On multiplication of 2 × 2 matrices. *Linear Algebra and its Applications*, 4(4):381–388, 1971.

[5] Volker Strassen. Gaussian elimination is not optimal. *Numerische Mathematik*, 13(4):354–356, 1969.

[6] Alexey V Smirnov. The bilinear complexity and practical algorithms for matrix multiplication. *Computational Mathematics and Mathematical Physics*, 53(12):1781–1795, 2013.

[7] Benjamin Lipshitz, Grey Ballard, James Demmel, and Oded Schwartz. Communication-avoiding parallel strassen: Implementation and performance. In *SC '12: Proceedings of the International Conference on High Performance Computing, Networking, Storage and Analysis*, pages 1–11, 2012.

[8] Sarah Quiñones. Nano-gemm. https://sarah-ek.veganb.tw/blog/nano-gemm/, 2026. Accessed: 16 July 2026.

[9] S. Huss-Lederman, E.M. Jacobson, J.R. Johnson, A. Tsao, and T. Turnbull. Implementation of strassen's algorithm for matrix multiplication. In *Supercomputing '96: Proceedings of the 1996 ACM/IEEE Conference on Supercomputing*, pages 32–32, 1996.

[10] Nicholas J. Higham. Exploiting fast matrix multiplication within the level 3 blas. *ACM Transactions on Mathematical Software (TOMS)*, 16(4):352–368, 1990.

[11] Paolo D'Alberto and Alexandru Nicolau. Using strassen's algorithm for dimensionality-independent matrix multiplication. *ACM Transactions on Mathematical Software (TOMS)*, 33(3):16–es, 2007.

[12] Jianyu Huang, Tyler M. Smith, Greg M. Henry, and Robert A. van de Geijn. Strassen's algorithm reloaded. In *SC16: International Conference for High Performance Computing, Networking, Storage and Analysis*, pages 690–701. IEEE Press, 2016.